

## Experiment No. 2

**Aim:** Create a Blockchain using Python

### Theory:

A **Blockchain** is a distributed and immutable digital ledger that stores data in a sequence of connected blocks. Each block contains information such as an index, timestamp, transactions, proof, and the hash of the previous block.

The connection between blocks is maintained using the **previous hash**. If the data in an earlier block is modified, its hash changes, making the subsequent blocks invalid.

### Main Components

#### 1. Block Creation

A block contains:

- Index
- Timestamp
- Transactions
- Proof
- Previous block hash

The first block in the blockchain is called the **Genesis Block**.

#### 2. Transactions

Transactions represent the data stored in a block. In this experiment, transactions contain:

- Sender
- Receiver
- Amount

Pending transactions are stored in a transaction list and are included when a new block is mined.

#### 3. Proof-of-Work

**Proof-of-Work (PoW)** is a computational process used for mining a block. The miner searches for a suitable proof/nonce that satisfies a cryptographic condition.

In this experiment, the required condition is that the SHA-256 hash must start with:

000

The experiment specifically requires modifying the original Proof-of-Work to use **three leading zeros**.

## Experiment No. 2

### 4. SHA-256 Hashing

SHA-256 is a cryptographic hashing algorithm used to generate a unique hash for each block. It helps maintain the integrity and immutability of the blockchain.

### 5. Blockchain Validation

The blockchain is considered valid when:

1. The **previous\_hash** of every block matches the actual hash of the previous block.
2. The Proof-of-Work condition is satisfied.

The **is\_chain\_valid()** method performs these checks.

### 6. Flask API

Flask is used to create a simple web API for interacting with the blockchain.

The implemented APIs are:

| API                     | Method | Purpose                   |
|-------------------------|--------|---------------------------|
| <b>/add_transaction</b> | POST   | Add a new transaction     |
| <b>/mine_block</b>      | GET    | Mine a new block          |
| <b>/get_chain</b>       | GET    | Display the blockchain    |
| <b>/is_valid</b>        | GET    | Check blockchain validity |
| <b>/check_proof</b>     | GET    | Verify Proof-of-Work      |

The supplied experiment requires Flask-based demonstration using API endpoints and Postman.

## Experiment No. 2

### Tools and Libraries Used

#### Software

- Python
- Flask
- Postman
- Command Prompt / Terminal

#### Python Libraries

- datetime
- hashlib
- json
- flask

### Algorithm / Steps

#### Step 1: Create Blockchain

Create a **Blockchain** class containing the blockchain and transaction list.

#### Step 2: Create Genesis Block

Initialize the blockchain with the first block called the Genesis Block.

#### Step 3: Add Transactions

Create a **create\_transaction()** method to store sender, receiver, and amount.

#### Step 4: Perform Proof-of-Work

Calculate the proof by repeatedly changing the nonce/proof until the SHA-256 hash starts with:  
**000**

#### Step 5: Mine Block

A block is mined only when the transaction list is not empty.

Before mining, the pending transactions are removed from the transaction list and added to the block.

#### Step 6: Calculate Hash

Generate the SHA-256 hash of the previous block.

#### Step 7: Add New Block

Create a new block containing the transactions, proof, timestamp and previous hash.

#### Step 8: Validate Blockchain

Check the previous hash and Proof-of-Work condition for every block.

**Experiment No. 2****Step 9: Create Flask APIs**

Create endpoints for adding transactions, mining, displaying the chain, and validating the blockchain.

**Step 10: Test Using Postman**

Send requests through Postman and verify the responses.

**Code:**

```
import datetime
import hashlib
import json
from flask import Flask, jsonify, request

class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        # Genesis Block
        self.create_block(
            proof=1,
            previous_hash='0',
            transactions=[]
        )

    def create_block(self, proof, previous_hash, transactions):
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'transactions': transactions,
            'proof': proof,
            'previous_hash': previous_hash
        }
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def create_transaction(self, sender, receiver, amount):
        transaction = {
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        }
        self.transactions.append(transaction)
        return len(self.chain) + 1

    def proof_of_work(self, previous_proof):
        new_proof = 1
```

## Experiment No. 2

```
check_proof = False
while check_proof is False:
    hash_operation = hashlib.sha256(
        str(
            new_proof ** 2 - previous_proof ** 2
        ).encode()
    ).hexdigest()
    # 000 leading zeros
    if hash_operation[:3] == '000':
        check_proof = True
    else:
        new_proof += 1
return new_proof

def hash(self, block):
    encoded_block = json.dumps(
        block,
        sort_keys=True
    ).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    if len(chain) == 0:
        return False
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        # Check previous hash
        if block['previous_hash'] != self.hash(previous_block):
            return False
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(
            str(
                proof ** 2 - previous_proof ** 2
            ).encode()
        ).hexdigest()
        # Check 000 leading zeros
        if hash_operation[:3] != '000':
            return False
        previous_block = block
        block_index += 1
    return True

# Create Blockchain
blockchain = Blockchain()

# Flask Application
app = Flask(__name__)
```

## Experiment No. 2

## # ADD TRANSACTION

```
@app.route('/add_transaction', methods=['POST'])
def add_transaction():
    json_data = request.get_json()
    sender = json_data.get('sender')
    receiver = json_data.get('receiver')
    amount = json_data.get('amount')
    if sender is None or receiver is None or amount is None:
        return jsonify({
            'message': 'Please provide sender, receiver and amount.'
        }), 400
    block_index = blockchain.create_transaction(
        sender,
        receiver,
        amount
    )
    response = {
        'message': 'Transaction added successfully.',
        'sender': sender,
        'receiver': receiver,
        'amount': amount,
        'block_index': block_index
    }
    return jsonify(response), 201
```

## # MINE BLOCK

```
@app.route('/mine_block', methods=['GET'])
def mine_block():
    # Mine only if transactions exist
    if len(blockchain.transactions) == 0:
        return jsonify({
            'message':
                'No transactions available. Add a transaction before mining.'
        }), 400
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    # Proof-of-Work
    proof = blockchain.proof_of_work(previous_proof)
    # Previous block hash
    previous_hash = blockchain.hash(previous_block)
    # Remove transactions before mining
    transactions_to_add = blockchain.transactions.copy()
    blockchain.transactions.clear()
    # Create block
    block = blockchain.create_block(
        proof,
        previous_hash,
        transactions_to_add
```

## Experiment No. 2

```
)
response = {
    'message': 'Congratulations, you just mined a block!',
    'index': block['index'],
    'timestamp': block['timestamp'],
    'transactions': block['transactions'],
    'proof': block['proof'],
    'previous_hash': block['previous_hash']
}
return jsonify(response), 200

# GET CHAIN

@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }
    return jsonify(response), 200

# CHECK VALIDITY

@app.route('/is_valid', methods=['GET'])
def is_valid():
    valid = blockchain.is_chain_valid(
        blockchain.chain
    )
    if valid:
        response = {
            'message': 'All good. The Blockchain is valid.'
        }
    else:
        response = {
            'message': 'The Blockchain is not valid.'
        }
    return jsonify(response), 200

# CHECK PROOF

@app.route('/check_proof', methods=['GET'])
def check_proof():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    hash_operation = hashlib.sha256(
        str(
            proof ** 2 - previous_proof ** 2
        ).encode()
    ).hexdigest()
```

## Experiment No. 2

```
response = {
    'previous_proof': previous_proof,
    'golden_nonce': proof,
    'hash': hash_operation,
    'required_condition': 'Hash must start with 000',
    'condition_satisfied':
        hash_operation.startswith('000')
}
return jsonify(response), 200

# RUN SERVER
if __name__ == '__main__':
    app.run(
        host='0.0.0.0',
        port=5000,
        debug=False
    )
```

## Output:

## 1. Flask Server Running:

```
C:\Users\INFT505-11\Desktop>python exp2bc.py
* Serving Flask app 'exp2bc'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.47.107:5000
Press CTRL+C to quit
```

Figure 1: Flask server running successfully and API requests being processed.

## 2. Add Transaction:

The screenshot shows a web browser interface for testing HTTP requests. The URL bar shows `http://127.0.0.1:5000/add_transaction`. The request method is set to **POST**. The request body is a JSON object:

```
{
  "sender": "Dipesh",
  "receiver": "Mayur",
  "amount": 10000000
}
```

The response status is **201 CREATED**, with a time of **4 ms** and size of **288 B**. The response body is a JSON object:

```
{
  "amount": 10000000,
  "block_index": 3,
  "message": "Transaction added successfully.",
  "receiver": "Mayur",
  "sender": "Dipesh"
}
```

Figure 2: Successfully adding a transaction using the Flask API.



## Experiment No. 2

## 3. Mine Block:

HTTP [http://127.0.0.1:5000/mine\\_block](http://127.0.0.1:5000/mine_block) Save

GET [http://127.0.0.1:5000/mine\\_block](http://127.0.0.1:5000/mine_block) Send

Params Authorization Headers (6) **Body** Pre-request Script Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary **JSON** Beautify

1

Body Cookies Headers (5) Test Results Status: 200 OK Time: 15 ms Size: 443 B Save Response

Pretty Raw Preview Visualize **JSON** ⌵ 🔍

```

1  {
2    "index": 3,
3    "message": "Congratulations, you just mined a block!",
4    "previous_hash": "171235d237e9ae267c55356139fedbdd75ce01b9a856c08ae83a47c2365520c0",
5    "proof": 11028,
6    "timestamp": "2026-09-02 12:04:34.124750",
7    "transactions": [
8      {
9        "amount": 10000000,
10       "receiver": "Mayur",
11       "sender": "Dipesh"
12     }
13   ]
14 }

```

Figure 3: Successfully mined a new block using Proof-of-Work.

## 4. Display Blockchain:

HTTP [http://127.0.0.1:5000/get\\_chain](http://127.0.0.1:5000/get_chain) Save

GET [http://127.0.0.1:5000/get\\_chain](http://127.0.0.1:5000/get_chain) Send

Params Authorization Headers (6) **Body** Pre-request Script Tests Settings Cookies

Query Params

| Key | Value | Bulk Edit |
|-----|-------|-----------|
| Key | Value |           |

Body Cookies Headers (5) Test Results Status: 200 OK Time: 4 ms Size: 737 B Save Response

Pretty Raw Preview Visualize **JSON** ⌵ 🔍

```

1  {
2    "chain": [
3      {
4        "index": 1,
5        "previous_hash": "0",
6        "proof": 1,
7        "timestamp": "2026-09-02 11:56:43.014734",
8        "transactions": []
9      }
10     ]
11 }

```

Figure 4: Displaying the blockchain using the /get\_chain API.

## Experiment No. 2

```
10 {
11   "index": 2,
12   "previous_hash": "fe53fb38a16dafa6edf17f44490fdd042efc721bb823bcf814306da23d32773f",
13   "proof": 533,
14   "timestamp": "2026-09-02 12:01:56.144796",
15   "transactions": [
16     {
17       "amount": 10000000,
18       "receiver": "Chirag",
19       "sender": "Dipesh"
20     }
21   ]
22 }
```

Figure 5: Continuation of the blockchain output.

```
23 {
24   "index": 3,
25   "previous_hash": "171235d237e9ae267c55356139fedbdd75ce01b9a856c08ae83a47c2365520c0",
26   "proof": 11028,
27   "timestamp": "2026-09-02 12:04:34.124750",
28   "transactions": [
29     {
30       "amount": 10000000,
31       "receiver": "Mayur",
32       "sender": "Dipesh"
33     }
34   ],
35   "length": 3
36 }
37 ]
38 }
```

Figure 6: Further details of the blockchain and transaction data.

## 5. Blockchain Validation:

```
1 {
2   "message": "All good. The Blockchain is valid."
3 }
```

Figure 7: Blockchain validation showing that the blockchain is valid.

## Experiment No. 2

## 6. Proof - of - Work Verification

HTTP [http://127.0.0.1:5000/check\\_proof](http://127.0.0.1:5000/check_proof) Save

GET [http://127.0.0.1:5000/check\\_proof](http://127.0.0.1:5000/check_proof) Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

| Key | Value | Bulk Edit |
|-----|-------|-----------|
| Key | Value |           |

Body Cookies Headers (5) Test Results Status: 200 OK Time: 4 ms Size: 355 B Save Response

Pretty Raw Preview Visualize JSON ⋮

```
1 {
2   "condition_satisfied": true,
3   "golden_nonce": 533,
4   "hash": "0000c00870f23a23ae80377298491b091db400d575be0efbde5b310f2f763ed1",
5   "previous_proof": 1,
6   "required_condition": "Hash must start with 000"
7 }
```

Figure 8: Verification of the Proof-of-Work condition with three leading zeros.

## 7. Proof API Request in Flask

```
C:\Users\INF505-11\Desktop>python exp2bc.py
* Serving Flask app 'exp2bc'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.47.107:5000
Press CTRL+C to quit
127.0.0.1 - - [02/Sep/2026 12:09:31] "GET /check_proof HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:09:41] "GET /check_proof HTTP/1.1" 200 -
```

Figure 9: Flask terminal showing successful execution of the Proof-of-Work verification API.

## Conclusion:

The basic concept of Blockchain was successfully implemented using Python. The experiment demonstrated transaction creation, block mining using Proof-of-Work, SHA-256 hashing, blockchain validation, and Flask-based API interaction. The blockchain was successfully verified as valid, demonstrating the basic principles of **immutability, hashing, and Proof-of-Work**.